The Islamia University of Bahawalpur

MujahidGPT ChatBot For JHK Solution

A project presented to

Department of Information Technology, IUB Bahawalpur

In partial fulfillment
of the requirement for the degree of

Bachelor of Science in Information Technology (2022-2026)

By

Name: Mujahid Hussain

Roll no: F22BINFT1M01235

Session: 2022-2026

DECLARATION

We hereby declare that this ChatBot, neither whole nor as a part has been copied out from any source. It is further declared that I have developed this ChatBot and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Student Name

Mujahid Hussain

CERTIFICATE OF APPROVAL

It is to certify that the Final Year Project of BS (IT) titled **"MujahidGPT"** was developed by **Mujahid Hussain**, Roll No. **F22BINFT1M01235**, under the supervision of **MR. MUZMAIL UR REHMAN**, and that in his opinion, it is fully adequate in scope and quality for the degree of **Bachelor of Science in Information Technology (BS-IT)**.

Supervisor

External Examiner

Chairman Department of Information Technology

Acknowledgement

First of all, I am thankful to Almighty Allah for giving me the strength, guidance, and ability to successfully complete this Final Year Project.

I would like to express my sincere gratitude to my respected supervisor, **MR. MUZMAIL UR REHMAN**, for his valuable guidance, continuous support, encouragement, and constructive suggestions throughout the development of this project. His supervision and motivation played an important role in the successful completion of this work.

I am also thankful to the faculty members of the Department of Information Technology for providing knowledge, assistance, and a positive learning environment during my academic journey.

Furthermore, I would like to thank my family and teachers for their support, motivation, and encouragement throughout the completion of this project.

Finally, I appreciate everyone who directly or indirectly contributed to the successful completion of the **"MujahidGPT"** project.

Mujahid Hussain

Abbreviations

Term	Definition
AI	A branch of computer science that enables machines and software systems to simulate human intelligence, including learning, reasoning, problem-solving, and decision-making. .
API	A set of rules and protocols that allows different software applications to communicate and exchange data with each other.
NLP	A field of Artificial Intelligence that focuses on enabling computers to understand, interpret, process, and generate human language.
LLM	An advanced AI model trained on large amounts of textual data to understand context and generate human-like responses in natural language.
UI	The visual and interactive components of a software application through which users communicate and interact with the system.
GUI	Graphical User Interface – a visual interface that allows users to interact with the system using graphical elements such as buttons, forms, and menus.
SRS	Software Requirements Specification – a document that defines the functional and non-functional requirements of the system.
SDD	Software Design Document – a document that describes the internal design, architecture, and implementation details of the system.
JSON	A lightweight and structured data-interchange format used for storing and transmitting information between applications and web services.
HTTPS	A secure communication protocol that encrypts data transmitted between a web browser and a server to ensure confidentiality and integrity.
GPT	A deep learning language model based on the Transformer architecture, designed to generate coherent and contextually relevant text responses

Table of Contents

📗 Software Design Document (SDD)..... 43

Chapter 1

Introduction

This chapter introduces the project background, objectives, scope, significance, and problem statement. MujahidGPT addresses client support challenges through automated AI-based communication. This section has been expanded for final year project reporting purposes and follows the information provided in the submitted SRS and SDD documents.
This chapter introduces the project background, objectives, scope, significance, and problem statement. MujahidGPT addresses client support challenges through automated AI-based communication. This section has been expanded for final year project reporting purposes and follows the information provided in the submitted SRS and SDD documents.
This chapter introduces the project background, objectives, scope, significance, and problem statement. MujahidGPT addresses client support challenges through automated AI-based communication. This section has been expanded for final year project reporting purposes and follows the information provided in the submitted SRS and SDD documents.
This chapter introduces the project background, objectives, scope, significance, and problem statement. MujahidGPT addresses client support challenges through automated AI-based communication. This section has been expanded for final year project reporting purposes and follows the information provided in the submitted SRS and SDD documents.
This chapter introduces the project background, objectives, scope, significance, and problem statement. MujahidGPT addresses client support challenges through automated AI-based communication. This section has been expanded for final year project report

📗 Software Requirement Specification (SRS)

1. Introduction

1.1 Purpose

This document specifies the software requirements for the Personal Assistant for JHK Solution, named MujahidGPT. It is a chatbot system designed to provide comprehensive information about JHK Solution's services, company details, and assist clients in obtaining services when company personnel are unavailable. This SRS covers the full scope of the system, including its core chatbot functionality, integration with APIs, and user interaction mechanisms. The revision number is 1.0, representing the initial release.

1.2 Document Conventions

This SRS follows standard typographical conventions: Bold text is used for section headings, italics for emphasis on key terms, and bullet points for lists of functions or requirements. Priorities for requirements are explicitly stated where applicable (High, Medium, Low), and higher-level requirements are inherited by sub-requirements unless otherwise noted. All requirements are numbered sequentially (e.g., REQ-1). "TBD" is used as a placeholder for any undecided elements.

1.3 Intended Audience and Reading Suggestions

This document is intended for developers, project supervisors (e.g., Mr. Muzamil ur Rehman), clients from JHK Solution, testers, and documentation writers. Developers should focus on sections 3, 4, and 5 for implementation details. Supervisors and clients may start with sections 1 and 2 for an overview, then proceed to section 4 for features. Testers should review section 5 for nonfunctional requirements. Read in sequence from section 1 for a complete understanding, skipping appendices unless needed for definitions.

1.4 Product Scope

MujahidGPT is a web-based personal assistant chatbot that enhances client engagement for JHK Solution by providing 24/7 access to company information and services. Benefits include improved customer satisfaction through instant responses, reduced workload on human staff, and alignment with JHK Solution's goal of efficient service delivery. Objectives are to handle inquiries reliably, ensure cross-platform accessibility, and integrate seamlessly with existing tools. This SRS does not cover hardware procurement or post-deployment maintenance beyond initial setup. For vision details, refer to the project proposal document.

1.5 References

- Hugging Face Documentation: https://huggingface.co/docs (for model hosting and integration).
- Gradio Documentation: https://gradio.app/docs (for UI building).
- Google Sites Guide: https://support.google.com/sites (for hosting).
- GitHub API Reference: https://docs.github.com (for version control).
- Groq API Documentation: Available at thenajafigroup.online (for API keys and usage).
- Python 3.10+ Standard Library: https://docs.python.org/3/.
- HTML5 and CSS3 Standards: https://www.w3.org/TR/html5/ and https://www.w3.org/Style/CSS/.
- IEEE SRS Template: IEEE Std 830-1998 (adapted for this project).

2. Overall Description

2.1 Product Perspective

MujahidGPT is a new, self-contained product developed as a final year project to serve as a virtual assistant for JHK Solution. It builds on existing chatbot technologies but is customized for JHK Solution's specific needs, such as providing company history, service listings, pricing, and contact details. Unlike general chatbots, it focuses on business-specific queries and acts as a fallback when staff are unavailable. The system interfaces with web APIs for real-time data and is hosted on platforms like Hugging Face Spaces. A high-level diagram would show: Client (user) -> Web Interface (Gradio) -> Chatbot Logic (Python) -> API Calls (Groq) -> Response Generation.

2.2 Product Functions

The major functions include:

- Handling natural language queries about JHK Solution's services, history, team, and contact information.
- Providing instant responses using pre-trained models and API integrations.
- Collecting user feedback for improvements.
- Logging interactions for analysis by JHK Solution admins.
- Supporting multi-language queries if expanded (initially English). A top-level data flow: User input -> NLP processing -> Database/API query -> Response output.

2.3 User Classes and Characteristics

- Clients: Frequent users seeking services; may have low technical expertise, expect simple interactions; high importance.
- Company Admins: Occasional users for monitoring; technical background, access to logs; medium importance.
- Developers/Supervisors: Infrequent users for testing; high technical expertise; low priority for satisfaction but critical for maintenance. Users are differentiated by privilege levels: Clients (read-only), Admins (read/write).

2.4 Operating Environment

The software operates on web browsers across Mac, Linux, and Windows platforms. Hardware: Standard desktop/laptop with internet access (no specific CPU/RAM requirements beyond browser capabilities). Operating systems: Any supporting modern browsers (Chrome, Firefox, Safari). It coexists with other web apps without conflicts, hosted on Hugging Face Spaces and Google Sites.

2.5 Design and Implementation Constraints

- Languages: Python for backend logic, HTML/CSS for frontend styling.
- Tools: Gradio for UI, Hugging Face for model hosting, GitHub for source control, Google Sites for deployment.
- APIs: Must use Groq API keys from thenajafigroup.online; no other AI APIs without approval.
- Constraints: No mobile app version (web-only); adhere to free-tier limits of hosting

platforms; ensure cross-browser compatibility; security protocols for API calls (HTTPS).
- Standards: Follow web accessibility guidelines (WCAG 2.1) for usability on all OS.

2.6 User Documentation

Delivered components: User manual (PDF), online help via in-chat prompts, and tutorials on GitHub README. Formats: Markdown for online, PDF for printable. Standards: Clear, step-by-step guides with screenshots.

2.7 Assumptions and Dependencies

Assumptions: Internet connectivity is always available for users; Groq API remains free/accessible; JHK Solution provides accurate company data for training. Dependencies: Groq API for NLP processing; Hugging Face/Gradio for hosting (project fails if platforms change terms); GitHub for version control. If assumptions change, requirements may need revision.

3. External Interface Requirements

3.1 User Interfaces

Logical characteristics: Web-based chat interface with text input box, response display area, and buttons for feedback (e.g., thumbs up/down). Follow Gradio UI standards: Simple layout, responsive design. Standard functions: Help button for FAQs, clear error messages (e.g., "Sorry, I didn't understand that"). Keyboard shortcuts: Enter to send message. Error display: Red text for invalid inputs. Details in separate UI spec document.

3.2 Hardware Interfaces

Supports standard web hardware: Keyboard/mouse for input, screen for output. No special devices; data interactions via HTTP. Supported types: Any device with browser (laptops, desktops). Protocols: Standard web protocols.

3.3 Software Interfaces

Connections: Python scripts interface with Gradio (version 3.x) for UI, Hugging Face libraries for models, Groq API (via HTTP requests). Data in/out: User queries (strings) in, responses (text) out. Services: API calls for NLP; shared data via session variables. Constraints: Use RESTful APIs; no global data areas.

3.4 Communications Interfaces

Requirements: Web browser for client-server communication; HTTPS for secure API calls. Message formatting: JSON for API responses. Standards: HTTP/HTTPS, no FTP. Security: API key encryption; transfer rates: <2s per response; synchronization: Real-time via websockets if implemented in Gradio.

4. System Features

Features organized by core services: Interaction, Retrieval, Handling, Feedback.

4.1 System Feature 1: Chatbot Interaction

4.1.1 Description and Priority

Interactive chat for user queries; High priority (benefit:9, penalty:9, cost:5, risk:3).

4.1.2 Stimulus/Response Sequences

User types query -> System processes -> Response displayed; Error if invalid -> Retry prompt.

4.1.3 Functional Requirements
REQ-1: System shall accept text inputs up to 500 characters.
REQ-2: System shall respond within 2 seconds using Groq API.

REQ-3: Handle errors gracefully with fallback messages.

4.2 System Feature 2: Information Retrieval

4.2.1 Description and Priority

Retrieve company info (services, contacts); High priority (benefit:8, penalty:7, cost:4, risk:2).

4.2.2 Stimulus/Response Sequences

Query like "What services does JHK offer?" -> Fetch data -> Display list.

4.2.3 Functional Requirements
REQ-4: Access pre-loaded company database.
REQ-5: Support keyword-based searches.

REQ-6: Update info via admin interface (TBD).

4.3 System Feature 3: Service Inquiry Handling

4.3.1 Description and Priority

Guide users on unavailable services; Medium priority (benefit:6, penalty:5, cost:3, risk:2).

4.3.2 Stimulus/Response Sequences

Inquiry when staff unavailable -> Provide alternatives or schedule.

4.3.3 Functional Requirements
REQ-7: Detect staff availability via API (TBD).

REQ-8: Suggest email/contact forms.

4.4 System Feature 4: User Feedback Collection

4.4.1 Description and Priority

Collect ratings/comments; Low priority (benefit:4, penalty:3, cost:2, risk:1).

4.4.2 Stimulus/Response Sequences

After response -> Prompt feedback -> Store.

4.4.3 Functional Requirements
REQ-9: Store feedback in log file.

REQ-10: Anonymize user data.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

System must handle 100 concurrent users with <2s response time. Rationale: Ensure fast access for clients. Specific: Real-time processing on standard hardware; individual features like retrieval <1s.

5.2 Safety Requirements

No direct harm risks, but prevent misinformation: Validate responses against company data. Refer to no external safety policies; no certifications needed. Actions: Log all interactions for audit.

5.3 Security Requirements

Protect API keys; use HTTPS. Authenticate admins via passwords. Refer to GDPR-like privacy; no certifications. User identity: Anonymous for clients.

5.4 Software Quality Attributes

Reliable (99% uptime), easy (intuitive UI), fast (low latency), accessible (cross-OS: Mac, Linux, Windows). Quantitative: Availability 99%, usability score >80% in tests. Preferences: Ease of use over learning curve. Other: Maintainable code, portable web app.

5.5 Business Rules

Admins can update info; clients read-only. Enforce: Only JHK-approved data displayed. Implies REQ-11: Role-based access.

6. Other Requirements

Database: Use lightweight JSON for storage. Internationalization: English only initially. Legal: Comply with data protection laws. Reuse: Leverage open-source Gradio components.

Appendix A: Glossary

- Chatbot: AI-driven conversational agent.
- Groq API: Fast AI inference service.
- Gradio: Python library for web UIs.
- NLP: Natural Language Processing.
- JHK Solution: The company this assistant serves.

Appendix B: To Be Determined List

1. Exact database schema for company info.
2. Multi-language support implementation.
3. Advanced analytics for logs.

Software Design Document (SDD)

MujahidGPT

1. Introduction

1.1 Purpose of the Document

This Software Design Document (SDD) describes the internal design, technical structure, and implementation approach of MujahidGPT. It explains how the system fulfills the requirements from the Software Requirements Specification (SRS), serving as a guideline for developers, testers, the supervisor (Mr. Muzamil ur Rehman), and future maintainers by detailing architecture, modules, data flows, integration points, and security/performance mechanisms. Unlike the SRS (focused on what the system must do), this document focuses on how it is built.

1.2 Scope of the System

MujahidGPT is a browser-based conversational AI assistant for JHK Solution clients. It handles natural language queries about company services, provides instant accurate information, guides on service requests when staff are unavailable, and collects feedback. The scope of this document covers: overall architecture, module/component design, data structures, Groq API integration, UI implementation, security, error handling, and performance/scalability. Functional and non-functional details are referenced from the SRS and not repeated here.

1.3 Definitions, Acronyms, and Abbreviations

- LLM: Large Language Model
- API: Application Programming Interface
- Groq API: Fast inference service for LLMs (compatible with OpenAI format)
- Gradio: Python library for building interactive web UIs
- Hugging Face Spaces: Platform for hosting ML demos
- JHK Solution: The target company/client
- NLP: Natural Language Processing
- SDD: Software Design Document

- SRS: Software Requirements Specification

1.4 References

- Groq API Documentation (console.groq.com/docs)
- Gradio Documentation (gradio.app/docs)
- Hugging Face Spaces Guide (huggingface.co/docs/hub/spaces)
- Python Official Documentation
- IEEE Std 1016-2009 (Software Design Descriptions)
- Groq Quickstart and Models Reference

1.5 System Overview

MujahidGPT operates as a client-server web application. Users interact via a Gradio-powered chat interface in the browser; Python backend processes inputs, constructs prompts with JHK knowledge, calls the Groq API for intelligent responses, formats outputs, and logs interactions/feedback. No heavy database is used—static JSON for knowledge, file-based logging. Hosting on Hugging Face Spaces ensures easy deployment and scaling.

2. Design Considerations

2.1 Design Goals

- Deliver fast, reliable responses (<2s)
- Ensure simple, intuitive chat UI
- Maximize cross-platform accessibility (Mac/Linux/Windows browsers)
- Minimize local compute via Groq offloading
- Support easy maintenance and academic demonstration
- Emphasize modularity and separation of concerns

2.2 Assumptions and Constraints

Assumptions: Stable internet, Groq API availability, accurate JHK data provided.

Constraints: Python/Gradio ecosystem only, free-tier hosting limits, no custom servers, reliance on external Groq API, browser-only access.

2.3 Development Environment

- Language: Python 3.10+
- UI: Gradio + HTML/CSS
- API: Groq (groq Python library)
- Hosting: Hugging Face Spaces
- Version Control: GitHub
- Testing: Local browser + Spaces preview

3. System Architecture Design

3.1 Architectural Overview

Adapted three-tier structure:

- Presentation: Browser-rendered Gradio chat UI
- Application: Python scripts (session handling, prompt engineering, API calls, formatting)
- Data: JSON files (knowledge) + file logs (conversations/feedback) External integration: Groq API endpoint for LLM inference.

Three-Tier Adapted Architecture Diagram (described): Browser → Gradio frontend → Python backend logic → Groq API (cloud) → response back; parallel static JSON load for knowledge.

Deployment Diagram (described): Client browser → Hugging Face Spaces (Gradio app container) → Groq cloud API; optional Google Sites for static info pages; GitHub repo for source.

3.2 Technology Stack

- Frontend: Gradio, HTML, CSS
- Backend: Python, groq library
- Data: JSON files
- Hosting: Hugging Face Spaces, Google Sites, GitHub

4. Module Design

4.1 Chat Interface Module

Gradio-based chat handles input textbox, message history display, typing indicator, feedback buttons (thumbs up/down).

4.2 Conversation Manager Module

Maintains session context, appends history to prompts, injects JHK knowledge/system instructions.

4.3 API Integration Module

Securely calls Groq chat completions endpoint, handles retries/errors, parses responses.

4.4 Knowledge Retrieval Module

Loads JSON company data, performs keyword/semantic matching to enrich prompts.

4.5 Feedback & Logging Module

Captures ratings/comments, appends to timestamped files for admin review.

5. Data Design

5.1 Data Overview

Lightweight file-based storage (no RDBMS) for simplicity and deployment ease. Follows structured format to ensure quick access/integrity.

Entity-Relationship Overview (described): CompanyInfo (static) → ConversationSession (ephemeral) → FeedbackEntry (logged).

Data Structure Diagram (described): JSON tree for knowledge; flat append logs for sessions/feedback.

5.2 Data Descriptions

- company_knowledge.json: Services, contacts, FAQs
- session_logs/: Timestamped conversation JSON
- feedback_logs/: User ratings/comments

6. Component/Class Design

6.1 Component Design Overview

Modular Python components (functions/classes) for reusability.

Hierarchy Diagram (described): Main app → ChatSession → KnowledgeBase + GroqClient + ResponseFormatter.

6.2 Component Responsibilities

- ChatSession: Manages history/context
- KnowledgeBase: Loads/queries static data
- GroqClient: API calls + error handling
- ResponseFormatter: Cleans/styles output

7. Sequence Design

7.1 Sequence Design Overview

Sequences show component interactions over time for key flows.

User Query Sequence Diagram (described): User → Gradio UI → Conversation Manager → Knowledge Retrieval → GroqClient → API → Formatter → UI display.

API Call & Response Sequence Diagram (described): Prompt construction → Groq call → parse → fallback if error.

Feedback Submission Sequence Diagram (described): Thumbs click → log append → thank you message.

8. User Interface Design

8.1 UI Design Overview

Single-page responsive chat: input box, bubbles (user blue, bot green), feedback icons. Focuses on simplicity and speed.

UI Navigation Flow Diagram (described): Chat home → optional help/about links.

8.2 Chat Interface

Text input, scrollable history, typing animation, clear error/feedback prompts.

9. Security Design

9.1 Security Design Overview

Protects API keys, prevents injection, anonymous access.

Authentication & Access Flow Diagram (described): No login; env vars for keys, HTTPS enforced.

9.2 Data Security

Keys in environment/secrets, HTTPS transmission, no sensitive user storage.

9.3 Input Validation

Length limits, sanitization against prompt attacks.

10. Error Handling and Validation Design

10.1 Error Handling Overview

Graceful fallbacks (static replies on API fail), user-friendly messages.

Error Handling Flow Diagram (described): Detect → log → fallback → inform user.

10.2 Validation Strategy

Query length, content checks, duplicate prevention in logs.

11. Performance and Scalability Design

11.1 Performance Design

Optimized prompts, fast Groq inference for <2s responses.

Request–Response Flow Diagram (described): Input → process → API → output.

11.2 Scalability Design

Hugging Face auto-scaling; modular for knowledge/model updates.

Scalability Expansion Diagram (described): Add models/knowledge without redesign.

12. Conclusion

This SDD provides a detailed, practical design for MujahidGPT, realizing SRS requirements through efficient, modern tools. It ensures reliable, fast, accessible implementation suitable for JHK Solution support and academic evaluation.

Note:

Functional/non-functional requirements are in the SRS. This document focuses on technical design and implementation.

13. References

[1] Groq API Reference, Groq Console, 2026.

[2] Gradio Documentation, gradio.app, 2026.

[3] Hugging Face Spaces SDK Guide, huggingface.co/docs, 2026.

[4] IEEE Std 1016-2009, Software Design Descriptions.

[5] Python Documentation, python.org.